

2026 九段公开赛

CSP-J 第 04 场 题解

入门级

时间：2026 年 9 月 12 日

九段 OI 训练平台 www.9duan.com

【目录】

档案核销 题解	3
题意理解	3
子任务 1: 照定义逐轮核销 (20 分, 测试点 01~04)	4
子任务 2: 全部核销 (10 分, 测试点 05~06)	4
子任务 3: 标签非降 (15 分, 测试点 07~09)	5
满分做法: 按字母分配核销名额 (55 分, 测试点 10~20)	6
易错点	7
叠片检测 题解	8
题意理解	8
子任务 1: 枚举选哪些样片 (10 分, 测试点 01~02)	9
子任务 2: 没有读数损耗 (15 分, 测试点 03~05)	10
子任务 3: 可行性动态规划 (30 分, 测试点 06~11)	10
满分做法: 升序扫描的贪心 (45 分, 测试点 12~20)	11
易错点	13
监控触发 题解	14
题意理解	14
子任务 1: 照定义逐次事件重算 (15 分, 测试点 01~03)	15
子任务 2: 只有一条记录 (10 分, 测试点 04~05)	16
子任务 3: 所有记录都有 $i = j$ (15 分, 测试点 06~08)	16
满分做法: 合并两张表求第 $need$ 小 (60 分, 测试点 09~20)	17
易错点	20
数据抢救 题解	21
题意理解	21
子任务 1: 枚举所有块的子集 (15 分, 测试点 01~03)	22
子任务 2: 只有一个数据卷 (15 分, 测试点 04~06)	23
子任务 3: 所有卷都没有数据块 (15 分, 测试点 07~09)	23
满分做法: 按卷分层的依赖背包 (55 分, 测试点 10~20)	24

易错点	26
-----------	----

档案核销 题解

【题意理解】

n 份档案按入库先后排成一列，编号依次是 $1, 2, \dots, n$ ，第 i 份带着一个小写字母标签 s_i 。标签的优先级从低到高依次是 **a** 到 **z**，所以「优先级最低」就是「字母最小」。系统要执行**恰好** k 次核销，每次的规则是：在当前尚未核销的档案里找出优先级最低的标签，若有多份档案带着这个标签，就核销入库最早、也就是队列中最靠前的那一份；其余档案的相对次序不动。问 k 次操作全部结束后，剩余档案的标签按原编号从小到大组成的字符串长什么样，全部核销完就输出 **EMPTY**。

读题时有两件事要先钉死。

第一，**核销不改变任何东西**。它不修改剩余档案的标签，也不打乱它们在队列里的次序。这意味着每份档案「何时被核销」在读入的一瞬间就已经定死了：先看标签的优先级谁低，标签一样再看编号谁小。把编号 i 的这份档案写成二元组 (s_i, i) ，它们两两不同，构成一个**全序**。

第二，既然是全序，那么「执行 k 次」这件事就等价于一句话：**把全序里最小的 k 份档案一次性核销掉**。这一步转化是全题的关键——它把一个「反复选、反复核销」的过程，变成了一个「选出一个集合」的静态问题，后面所有做法都建立在它上面。

为什么这个转化是对的？用归纳法看一眼：第一次操作核销的就是全序最小的那一份；做完之后剩下的档案集合，其内部的相对全序和原来完全一致（因为标签和编号都没变），所以第二次操作核销的就是全序第二小的那一份，依此类推。

拿样例 1 走一遍。 $s = \text{cccaabababaccbc}$ ， $k = 3$ 。标签 **a** 出现在编号 4, 5, 7, 9, 11 上，是优先级最低的一档；全序最小的三份就是编号 4, 5, 7。

原队列	c	c	c	a	a	b	a	b	a	b	a	c	c	b	c
编号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
灰底加一条斜线的格子表示这一份被核销，格子里的标签仍然保留。															
全序最小的三份是 $(a, 4) < (a, 5) < (a, 7)$ 。															
核销后	c	c	c	b	b	a	b	a	c	c	b	c	← 输出		
原编号	1	2	3	6	8	9	10	11	12	13	14	15			

图 1：样例 1 的核销过程：上一行是原队列，灰底加斜线的第 4, 5, 7 格就是全序最小的三份档案；把它们抽走、其余按原编号合拢成下一行，下一行每个格子下方标的是它在原队列中的编号，读出来就是 **cccbabaccbc**。

注意剩下的串里 **a** 还有两份（原编号 9 和 11），**b** 一份没少——因为 k 只有 3，还轮不到 **b**。四个子任务的区别，就在于用什么办法找出「全序最小的 k 份」。

【子任务 1：照定义逐轮核销（20 分，测试点 01~04）】

这一档只保证 $n \leq 2000$ ，那就直接把题目的操作原样写出来：开一个 `removed` 数组标记谁已经被核销，做 k 轮，每轮从头到尾扫一遍，找出仍在队列里的档案中标签最小的那一份。

```

1  vector<int> removed(n, 0);
2  // 每一轮都精确地找出最小标签，以及它最靠左的那个位置。
3  for (int step = 0; step < k; ++step) {
4      int best = -1;
5      for (int i = 0; i < n; ++i) {
6          if (!removed[i] && (best == -1 || s[i] < s[best])) best =
        ↪ i;
7      }
8      removed[best] = 1;
9  }
10 // 未被删除的位置保持原来的先后顺序。
11 string answer;
12 for (int i = 0; i < n; ++i) if (!removed[i]) answer += s[i];
13 if (answer.empty()) cout << "EMPTY\n";
14 else cout << answer << '\n';

```

有一处细节值得看清楚：比较写的是 $s[i] < s[best]$ ，用的是**严格小于**。因为 i 是按编号从小到大枚举的，靠前的档案会先坐上 `best`，后面标签相同的档案不满足严格小于，抢不走它。所以「并列取最靠前」这条规则，是靠**严格小于加上从前往后枚举**自动实现的，不需要额外写编号比较。如果这里手滑写成 $\leq$ ，`best` 就会被同标签的更靠后的档案不断顶掉，核销的变成最靠后那份，样例 1 立刻就错。

复杂度是 $O(nk)$ 。 $n = k = 2000$ 时约 4×10^6 次比较，一秒之内轻松跑完；但完整范围下 $n = k = 4 \times 10^5$ ，就是 1.6×10^{11} 次，差着五个数量级，必然超时。

为什么不能推广：它每一轮都把整列档案重新扫一遍，可前一轮扫出来的信息（谁是次小、谁是第三小）全被丢掉了。 k 轮之间没有任何复用，这是 $O(nk)$ 的根源。

【子任务 2：全部核销（10 分，测试点 05~06）】

这一档给的特殊性质是 $k = n$ 。一次操作恰好核销一份，做 n 次自然一份不剩，所以不管 s 长什么样，答案都是 `EMPTY`。

```

1  int n, k;
2  string s;
3  cin >> n >> k >> s;
4  // 全部核销时，答案就是一个空序列。

```

```

5     if (k == n) cout << "EMPTY\n";
6     else cout << "invalid-erase-all-tier\n";

```

这一档看着像送分，但它其实是在提醒一条容易漏的边界：没有剩余档案时要输出 **EMPTY**，而不是输出一个空行。写满分做法时如果直接 `cout << answer`，在 $k = n$ 的数据上就会交出一个空串，判错。样例 3 正是这个边界最小的样子：只有一份标签为 u 的档案， $k = 1$ ，一次操作把它核销掉，输出的就是 **EMPTY**。

复杂度 $O(n)$ （读入占的），额外空间 $O(1)$ 。

为什么不能推广： $k < n$ 时会有档案留下，留下的是哪些完全取决于 s ，这一档一个字都没看 s 。

【子任务 3：标签非降（15 分，测试点 07~09）】

这一档保证 s 从前到后非降。此时全序 (s_i, i) 和编号序 i 是同一个序：标签本来就不减，标签相同时按编号排——所以编号 i 的档案，其全序名次就是 i 。

于是「核销全序最小的 k 份」退化成「核销最靠前的 k 份」，答案就是从第 $k + 1$ 份开始的后缀，一句 `substr` 解决。

```

1     bool sorted = true;
2     for (int i = 1; i < n; ++i) if (s[i] < s[i - 1]) sorted = false;
3     // 标签非降时，删除优先级恰好就是下标顺序。
4     if (sorted) {
5         string answer = s.substr(k);
6         if (answer.empty()) cout << "EMPTY\n";
7         else cout << answer << '\n';
8     } else cout << "invalid-sorted-tier\n";

```

复杂度 $O(n)$ ，除输入串和 `substr` 造出的答案串之外只用 $O(1)$ 的额外空间。

为什么不能推广：一旦 s 不是非降的，优先级最低的标签就可能藏在队列中间，最靠前的 k 份里可能混着大标签。样例 1 就是反例： s 以三个 c 开头，直接核销最靠前的 3 份会把三个 c 一并核销，输出 `aabababaccbc`，而正确答案是 `cccbbabaccbc`。这种一律砍前 k 份的写法，除了本档之外还会在 $k = n$ 的测试点 05~06 上恰好蒙对（答案同为 **EMPTY**），合计至多 25 分。

【满分做法：按字母分配核销名额（55 分，测试点 10~20）】

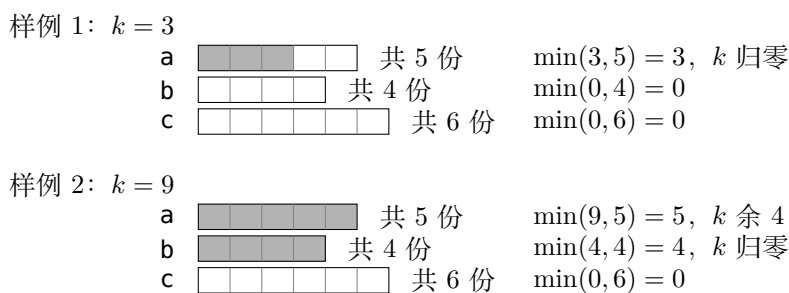
要在 $O(n)$ 内选出全序最小的 k 份，得先注意到一件事：**标签只有 26 种**。全序是「先按标签、再按编号」，所以被核销的这 k 份一定长成这个样子——把标签 **a** 的档案从前往后核销若干份，取完了再核销 **b** 的若干份，依此类推；而且至多只有一种标签是「取一半」的，它前面的标签全被取光，它后面的标签一份没动。

于是只要知道每种标签**各要核销几份**就够了，具体是哪几份可以事后再定。先数出每种标签的总份数，再从 **a** 到 **z** 依次分配名额，能拿多少拿多少，直到 k 用完：

```
1 // cnt[c] 表示标签 c 还要核销掉最靠前的几份。
2 int cnt[26] = {0};
3 for (int i = 0; i < n; ++i) cnt[s[i] - 'a']++;
4 for (int c = 0; c < 26; ++c) {
5     int take = min(k, cnt[c]);
6     cnt[c] = take;
7     k -= take;
8 }
```

这段代码有个很省事的写法：循环结束后， $\text{cnt}[c]$ 已经不再是「标签 c 有几份」，而被就地改写成了「标签 c 还要核销几份」。 $\text{take} = \min(k, \text{cnt}[c])$ 一句同时管住了两种情况—— k 有余就把这种标签全核销掉， k 不够就只核销 k 份，之后 k 归零，后面所有标签分到的都是 0。

下面这张图把两个样例的分配过程画在一起，正好能看清名额是怎么跨字母溢出去的。



每一小格代表一份档案：深色是分到的核销名额，浅色是保留下来的档案。

图 2: 两个样例共用同一串档案 **cccaabababaccbc**，其中标签 **a** 有 5 份、**b** 有 4 份、**c** 有 6 份。样例 1 的 $k = 3$ 全落在 **a** 上，剩下的串是 **cccbababaccbc**；样例 2 的 $k = 9$ 先取光 **a** 的 5 份，余下 4 份恰好取光 **b**，两次都没动到 **c**，剩下的串是 **cccccc**。

图里样例 2 那一组正是「跨字母」的典型： $k = 9$ 比 **a** 的 5 份多，多出来的 4 份名额就溢到 **b** 上，而 **b** 恰好也是 4 份，于是 **a** 和 **b** 双双清空，**c** 的 6 份原封不动。若把 k 再调小一点，比如设成 7，那么 **a** 取光后只剩 2 个名额落到 **b** 上，**b** 就成了唯一被切开的那种标签——这正是前面那条结构性观察。

名额定下来之后，再从前到后扫一遍原串，把名额消耗掉：

```
1 // 从左往右扫描，删掉的自然就是每种标签最靠前的那几次出现。
2 string answer;
3 for (int i = 0; i < n; ++i) {
4     int c = s[i] - 'a';
5     if (cnt[c] > 0) cnt[c]--;
6     else answer += s[i];
7 }
8 if (answer.empty()) cout << "EMPTY\n";
9 else cout << answer << '\n';
```

为什么这样扫出来的核销集合，和逐轮操作核销的**完全是同一批档案**？因为对每一种标签 c ，从前往后扫时最先遇到的 $\text{cnt}[c]$ 份，正是标签为 c 的档案里编号最小的那几份——而全序在同标签内部比的就是编号。所以不同标签之间由分配环节保证正确，同标签内部由从前往后扫描保证正确，两件事合起来不重不漏。

复杂度 $O(n + 26)$ ，也就是线性。空间上除了输入串，主要是答案串的 $O(n)$ 。 $n = 4 \times 10^5$ 时份数和编号都在 `int` 范围内，不需要 `long long`。

最后那句 `if (answer.empty())` 就是子任务 2 提醒过的边界： $k = n$ 时答案串为空，必须打印 `EMPTY`。

【易错点】

- **同标签必须核销最靠前的那份。**逐轮扫描时比较写成 `<=`，会把 `best` 顶到同标签更靠后的档案上；满分做法里如果按标签顺序输出而不是按原编号扫描，也是同一类错误。样例 1 上立刻能验出来。
- **直接核销最靠前的 k 份。**只在标签非降那一档成立。样例 1 里它输出 `aabababaccbc`，与正确答案 `ccbbabaccbc` 差得很远，这种写法至多拿 25 分。
- **忘了 `EMPTY`。** $k = n$ 时答案为空串，直接输出会交出一个空行。样例 3 和测试点 05~06 就是专门盯这个的。
- **把 `cnt` 的含义搞混。**分配循环之后 `cnt[c]` 已经从「有几份」变成了「还要核销几份」，如果在第二遍扫描里又拿它当份数用，会把整列档案核销光。
- **只减 k 不减 `cnt` (或反过来)。**`take` 必须同时写回 `cnt[c]` 并从 k 里扣掉，漏掉任何一边都会让名额对不上，核销的份数不等于 k 。
- **多试几种 k 。** $k = 1$ 、 $k = n - 1$ 、 $k = n$ 这三个边界最容易出问题，建议本地都跑一遍再交。

叠片检测 题解

【题意理解】

N 片待测样片、 M 个检测匣。第 i 片样片的基准读数是 S_i ；把它放进某个检测匣以后，若它的前方还有 r 片样片（也就是它位于深度 r ），读数就要打一道折扣，有效读数变成 $\max(0, S_i - rD)$ ，只有有效读数不小于仪器阈值 L 才能纳入检测。样片放进哪个检测匣、匣内次序怎么排都由我们说了算，问最多能纳入多少片。

把题目翻译一遍，最要紧的是**换一个视角看「位置」**。题面说的是「前方有 r 片样片」，我们把它反过来看：每个检测匣从前到后就是深度 0、深度 1、深度 2……这样一串位置，放在深度 r 上的样片需要 $S_i \geq L + rD$ （数据范围保证 $L \geq 1$ ， $\max$ 里的那个 0 永远够不着阈值，所以这一步转换是等价的）。于是每个位置都自带一个**门槛**，这个门槛与放进去的是哪一片样片无关。样例 1 的解释里说「深度 0, 1, 2 的位置分别要求基准读数至少为 5, 6, 7」，讲的正是这件事。

再往前一步： M 个检测匣的深度 0 位置门槛都是 L ，深度 1 位置门槛都是 $L + D$ ……也就是说，**门槛只跟深度有关，跟是哪个检测匣无关**。既然如此，检测匣的编号就完全不重要了，我们干脆把所有位置按门槛从小到大排成一条队：前 M 个位置门槛都是 L ，接下来 M 个都是 $L + D$ ，再接下来 M 个都是 $L + 2D$ ……第 j 个位置（ j 从 0 数起）的门槛就是

$$\text{need}(j) = L + \left\lfloor \frac{j}{M} \right\rfloor D.$$

这一步把二维的「检测匣 $\times$ 深度」压成一维的门槛序列，下面四档做法都建立在它上面。

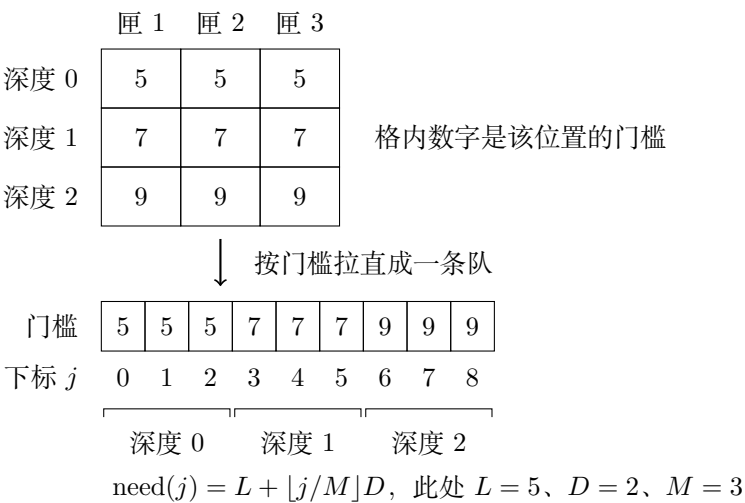


图 3: $M = 3$ 、 $D = 2$ 、 $L = 5$ 时的位置门槛。上半部分是原始的「检测匣 $\times$ 深度」网格，同一深度的 M 个位置门槛完全相同，所以检测匣的编号可以丢掉；下半部分是拉直后的一条不减门槛序列，下标 j 从 0 数起，方括号标出每三个位置来自同一深度。

有了门槛序列，题目就变成了一个纯粹的匹配问题：从 N 个基准读数里挑出尽量

多的，一一放进门槛序列的位置上，使每个位置上的基准读数都不小于它的门槛。四个子任务的区别，就在于用什么办法解这个匹配。

还有一个小结论要先记下：如果最终纳入了 x 片样片，那么把它们放进**最浅的 x 个位置**一定是最划算的——门槛序列不减，往深处挪只会让要求更严。所以永远不必考虑「跳过一个浅位置去用深位置」。

【子任务 1：枚举选哪些样片（10 分，测试点 01~02）】

这一档 $N, M \leq 5$ ，样片最多 5 片，子集只有 $2^5 = 32$ 个，可以全枚举。

对每个子集，把选中的基准读数升序排好，第 i 小的那片就去对第 i 个位置（ i 从 0 数起），逐个检查是否够门槛，全都够就说明这个子集可行：

```

1  int best = 0;
2  // 每个选出的子集，都拿去和同样数量的最浅槽位做匹配。
3  for (int mask = 0; mask < (1 << n); ++mask) {
4      vector<long long> chosen;
5      for (int i = 0; i < n; ++i) if (mask & (1 << i)) chosen.
        ↪ push_back(s[i]);
6      sort(chosen.begin(), chosen.end());
7      bool ok = true;
8      for (int i = 0; i < (int)chosen.size(); ++i) {
9          long long need = l + (long long)(i / m) * d;
10         if (chosen[i] < need) ok = false;
11     }
12     if (ok) best = max(best, (int)chosen.size());
13 }
14 cout << best << '\n';

```

这里「升序基准读数对升序门槛」的配法为什么是最优的？因为两边都是不减序列，如果某个可行方案里有两片样片的基准读数一大一小、却把大的放在浅位置、小的放在深位置，那把它俩换过来仍然可行（小的换到更松的位置，大的换到更严的位置但它本来就更大）。反复交换下去必然变成升序对升序，所以只要检查升序这一种配法就够了，不会漏掉可行子集。

复杂度 $O(2^N \cdot N \log N)$ 。 $N = 5$ 时不过几百次操作； $N = 50000$ 时 2^N 是天文数字。

为什么不能推广：它把「选哪些样片」当成一个个独立的整体去试，完全没有利用「门槛不减」这个结构。

【子任务 2：没有读数损耗（15 分，测试点 03~05）】

特殊性质 A 保证 $D = 0$ ，也就是每往深处放一层都不产生读数损耗。这时门槛序列退化成常数：每个位置的门槛都是 L 。位置又多得用不完（每个检测匣都能一直往深处放， N 片样片总安置得下），于是每片样片只要自己 $S_i \geq L$ 就一定能纳入检测，彼此之间毫无竞争。

```

1  int ans = 0;
2  for (int i = 0; i < n; ++i) {
3      cin >> x;
4      if (x >= l) ans++;
5  }
6  // 损耗为零时，每个位置的阈值都是同一个 L。
7  if (d == 0) cout << ans << '\n';
8  else cout << -11 << '\n';

```

所以答案就是满足 $S_i \geq L$ 的样片片数，边读边数即可，连排序都不用。复杂度 $O(N)$ ，空间 $O(1)$ 。代码最后一行的 -11 是给非本档数据准备的占位输出：这份程序只对 $D = 0$ 的测试点负责，一旦读到 $D > 0$ 就输出一个不可能成为答案的数，免得在别的测试点上碰巧得分。

为什么不能推广： $D > 0$ 时深处位置的门槛会抬高，浅位置成了稀缺资源，样片之间开始互相挤占。这时若仍旧只拿 S_i 跟 L 比较，就会把那些「够得着 L 、却挤不进浅位置」的样片也算进去，答案偏大——样例 1 的 $D = 1$ ，这种只比 L 的写法会把三片样片全部数上，而正确答案是 2，这样的程序也就只能拿到这一档的 15 分。

【子任务 3：可行性动态规划（30 分，测试点 06~11）】

把基准读数升序排好、门槛序列算出来之后，用一个布尔型的动态规划来做匹配。设 $dp[j]$ 表示「最浅的 j 个位置能不能被填满」，初始 $dp[0]$ 可行。逐片处理样片，倒序枚举 j ：若 $dp[j]$ 可行，且这片样片的基准读数够得上第 j 个位置的门槛，就把 $dp[j+1]$ 也标成可行。

```

1  sort(s.begin(), s.end());
2  for (int j = 0; j < n; ++j) need[j] = l + (long long)(j / m) * d;
3  vector<int> dp(n + 1, -1000000);
4  dp[0] = 0;
5  // dp[j] 记录前 j 个阈值槽位能否被填满。
6  for (int i = 0; i < n; ++i) {
7      for (int j = n - 1; j >= 0; --j) {
8          if (dp[j] >= 0 && s[i] >= need[j]) dp[j + 1] = max(dp[j + 1], dp[j] + 1);

```

```

9         }
10    }
11    int ans = 0;
12    for (int j = 0; j <= n; ++j) if (dp[j] >= 0) ans = j;
13    cout << ans << '\n';

```

两个实现细节。其一， j 必须**倒序**枚举，否则同一片样片会在一轮里被连着用好几次，答案偏大——这和 01 背包倒序更新是同一个道理。其二，代码用 -10^6 当「不可行」的标记， $dp[j] \geq 0$ 就表示可行；实际上可行时 $dp[j]$ 恒等于 j ，所以它本质上是可行性数组。

因为基准读数已经升序，第 i 片只会被用来填第 j 个位置且 $j \leq i$ ，所以「升序对升序」这个最优配法被完整地覆盖了，不会漏解。

复杂度 $O(N^2)$ ： $N = 2000$ 时约 4×10^6 次内层判断，两秒绰绰有余； $N = 50000$ 时是 2.5×10^9 次，稳超时。

为什么不能推广：瓶颈全在内层对 j 的那层循环上。每读一片样片都要把长度 $O(N)$ 的 dp 数组整个扫一遍，可这一遍里真正被改动的格子至多只有一个，其余判断全是白做。

【满分做法：升序扫描的贪心（45 分，测试点 12~20）】

上一档的 dp 数组里，可行的 j 其实永远是一段前缀 $0, 1, \dots, ans$ ：因为门槛不减，能填满 j 个位置就一定能填满更少的位置。既然可行集合就是一段前缀，那记住这段前缀的长度 ans 就够了，整个数组都可以扔掉。

于是做法变成：把基准读数升序排序，从小到大一片一片看。当前已经填好了 ans 个位置，下一个待填位置的门槛就是 $L + \lfloor ans/M \rfloor D$ 。够得上就把这片样片放进去、 ans 加一；够不上就**丢掉这片样片**， ans 不动。

```

1    vector<long long> s(n);
2    for (int i = 0; i < n; ++i) cin >> s[i];
3    sort(s.begin(), s.end());
4
5    // 已分配的 ans 片样片，占的正是所有检测匣里最浅的 ans 个槽位。
6    int ans = 0;
7    for (int i = 0; i < n; ++i) {
8        long long need = l + (long long)(ans / m) * d;
9        if (s[i] >= need) ans++;
10   }
11   cout << ans << '\n';

```

这里有两个地方值得停下来想清楚。

其一，够得上就立刻用，会不会浪费掉一片「本该留给更深位置」的大读数？不会。设想某个最优方案没有用当前这片 s_i ，而是用了另一片 s_t 来填这个位置。因为我们是升序扫描、 s_i 是当前还没被用掉的最小基准读数，所以 $s_t \geq s_i$ 。把两者对调： s_i 放到这个位置上，它本来就够门槛； s_t 放到 s_t 原先的位置上（如果 s_t 原先没被用，那就直接空着）。 $s_t \geq s_i$ ，所以对调后每个位置仍然满足门槛，纳入的片数不变。反复对调，就能把最优方案改造成贪心的选法，说明贪心不比最优差。

其二，够不上就直接丢，会不会丢错？不会。 s_i 是当前最小的未用基准读数，它连最浅的空位都够不上；而其余空位的门槛只会更高、更够不上。所以 s_i 在这一刻就已经彻底没用了。注意这里丢掉的是样片，不是位置——如果反过来写成「跳过这个位置去试下一个位置」，门槛只会变大，那是死路。这是本题实现中最容易写反的一处。

拿样例 1 把整个扫描过程走一遍。 $N = 3$ 、 $M = 1$ 、 $D = 1$ 、 $L = 5$ ，三片样片的基准读数依次是 5, 7, 5，排序后为 5, 5, 7。

步骤	第 1 步	第 2 步	第 3 步
基准读数 (升序)	5	5	7
当前门槛	$5 + \lfloor 0/1 \rfloor \cdot 1 = 5$	$5 + \lfloor 1/1 \rfloor \cdot 1 = 6$	$5 + \lfloor 1/1 \rfloor \cdot 1 = 6$
结果	$5 \geq 5$ ，放入	$5 < 6$ ，丢弃	$7 \geq 6$ ，放入
已填 ans	$0 \rightarrow 1$	$1 \rightarrow 1$	$1 \rightarrow 2$

最终 $ans = 2$ ：基准读数 5 与 7 的两片占据深度 0 与 1，另一片 5 无处可放。

图 4：样例 1 的贪心扫描，三列依次对应排序后扫到的三片样片。第 2 步样片被丢弃后 ans 不变，所以第 3 步面对的门槛仍然是 6——门槛只随「已填位置数」增长，不随「看过几片样片」增长。

图里第 2 步和第 3 步的门槛都是 6，这一点最能说明问题：丢掉一片样片并不会推高门槛，推高门槛的只有「成功放入」。如果把 ans 写成了循环变量 i ，门槛就会随着看过的片数一起涨，样例 1 里第三片面对的门槛会变成 $5 + \lfloor 2/1 \rfloor \cdot 1 = 7$ ， $7 \geq 7$ 侥幸也能过，但把这片的基准读数换成 6 就会算错。

复杂度：排序 $O(N \log N)$ ，扫描 $O(N)$ ，合计 $O(N \log N)$ 。 $N = 5 \times 10^4$ 时排序不过几十万次比较，远低于两秒。空间 $O(N)$ ，就是那个基准读数数组。

数值范围上要留意： $\lfloor ans/M \rfloor$ 最大约 5×10^4 ，乘上 $D \leq 5000$ 就是 2.5×10^8 ，再加 L 仍在 int 内，但代码把 D 、 L 和基准读数都声明成了 `long long`，并且在乘法处显式写了 `(long long)`，这是稳妥的写法——把边界放宽一点，比事后查溢出省事得多。

【易错点】

- **深度从 0 开始。**最前方的位置不打折扣，门槛就是 L 。把公式写成 $L + \lfloor j/M \rfloor D$ 时 j 必须从 0 数起，写成从 1 数起会让答案凭空少一层。
- **等号成立也算达标。**题面写的是「有效读数不小于仪器阈值 L 」，比较必须用 $\geq$ 。样例 1 里第一片恰好是 $5 = L$ ，写成 $>$ 立刻少算一片。
- **丢的是样片不是位置。**基准读数不够时只能跳过这片样片，绝不能跳到更深的位置去试——门槛只会更高。
- **门槛跟着 ans 走，不跟着 i 走。**用循环下标算深度是最常见的错法，它把被丢弃的样片也算成占了位置。
- **动态规划要倒序更新。**正序会让同一片样片被重复使用，答案偏大。
- **$D = 0$ 不是特例，是普通情形。**满分做法在 $D = 0$ 时门槛恒为 L ，扫描自然给出正确答案，不需要为特殊性质 A 单独写分支。

监控触发 题解

【题意理解】

N 个计数器全部从 0 开始，只会增加、永不减少。 Q 条监控记录，每条是一个三元组 (i, j, k) ，当 $h_i + h_j \geq a_k$ 时它处于触发状态（ $i = j$ 时左边就是 $2h_i$ ）。 K 次事件，每次给某个计数器加 1，然后要输出当前处于触发状态的记录条数。

题面里有一句话是整道题的出发点：**计数器只会增加，所以已经触发的记录不会恢复为未触发**。这意味着每条记录的状态只有两段：先是一直不触发，从某一次事件起一直触发。于是每条记录都可以用一个数字完全刻画：它的首次触发事件号。

有了这个观察，问题立刻换了个样子：

- 原问题： K 次事件里，每次事件之后问「现在有多少条记录处于触发状态」；
- 新问题：先给每条记录算出首次触发事件号 d_z （永不触发的记作 $+\infty$ ），再对每个 d 数一数有多少个 $d_z \leq d$ 。

后半句只要开一个桶 `born[d]` 数「第 d 次事件之后首次触发的有几条」，再做一遍前缀和即可，是 $O(K)$ 的。所以**全部难度都被压缩到了「求一条记录的首次触发事件号」这一件事上**。四个子任务的区别，正是求这个编号的手段。

再把 h_i 写清楚一点。设 $days[x]$ 是「计数器 x 被增加的那些事件编号」，按升序排好（读入时天然就是升序）。那么第 d 次事件结束时，

$$h_x(d) = |\{t \in days[x] : t \leq d\}|.$$

也就是说 $h_x(d)$ 就是 $days[x]$ 里不超过 d 的元素个数。

用样例把这些量都摆出来。 $N = 5$ 、 $M = 3$ 、 $K = 4$ 、 $Q = 4$ ，阈值 $a = (4, 3, 1)$ ，事件序列是 2, 2, 5, 1，于是 $days[1] = \{4\}$ ， $days[2] = \{1, 2\}$ ， $days[5] = \{3\}$ ， $days[3] = days[4] = \emptyset$ 。

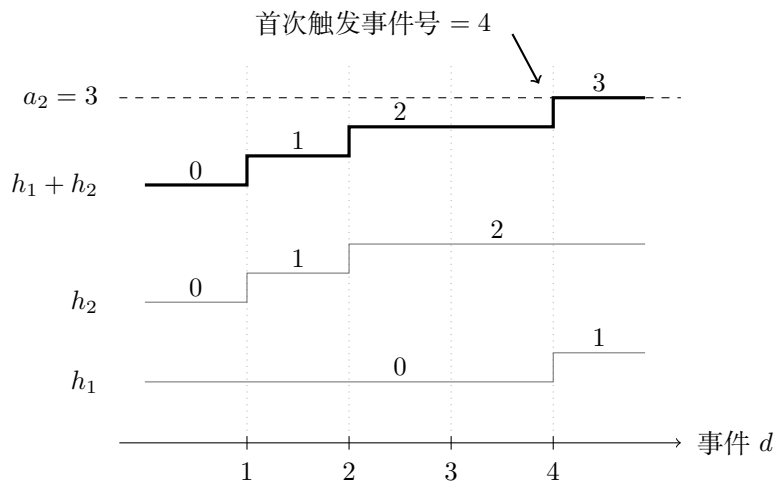


图 5: 样例中记录 (1,2,2) 的三条阶梯, 最上面那条粗线是两者之和, 两条细线是各自的计数器。竖直点线标出四次事件的位置, 每段折线上方的数字是这一段里的取值。 h_1 只在第 4 次事件抬一级, h_2 在第 1、2 次事件各抬一级; 和第一次够到虚线 $a_2 = 3$ 正是在第 4 次事件。

图里那条粗阶梯有个性质要留意: **它每次事件最多抬高一级** (一次事件只加一个计数器)。所以「和」第一次达到 a 的那次事件, 正好就是它第 a 次抬级的那次事件。这句话马上会变成满分做法的核心。

【子任务 1: 照定义逐次事件重算 (15 分, 测试点 01~03)】

这一档 N, M, K, Q 全都不超过 30, 直接把定义抄下来: 维护数组 h , 每次事件先把 h_{x_d} 加一, 再把 Q 条记录挨个判一遍。

```

1 // h[x] 是当前事件结束后计数器 x 的值, 初始全部为 0。
2 vector<int> h(n + 1, 0);
3 // 这一小规模做法直接扫描, 不需要完整范围中的二分查找与前缀和。
4 for (int d = 1; d <= k; d++) {
5     h[event[d]]++;
6     int answer = 0;
7     // 每轮逐条执行定义中的判断, 重复记录也各计一次。
8     for (int z = 0; z < q; z++) {
9         if (h[u[z]] + h[v[z]] >= a[t[z]]) answer++;
10    }
11    // 边界与答案: 当天更新必须先发生, 随后才输出本轮计数。
12    cout << answer << '\n';
13 }
```

题面特意强调了「每一行记录都独立计数, 即使三元组完全相同也算不同记录」, 这份代码天然满足——它是按行遍历的, 重复的行自然被数了多次。想着「去个重能省

点事」反而会算错。

复杂度 $O(KQ)$ 。 $K = Q = 30$ 时不过 900 次判断；完整范围下 $K = Q = 2 \times 10^5$ ，就是 4×10^{10} 次，差了七个数量级。

为什么不能推广：每次事件都把全部记录重新判一遍，可绝大多数记录的状态跟上一次事件比根本没变。真正变化的只有那些「本次事件恰好跨过门槛」的记录。

【子任务 2：只有一条记录（10 分，测试点 04~05）】

特殊性质 A 保证 $Q = 1$ 。这时候每次事件后的答案只有 0 和 1 两种，直接维护 $h_u + h_v$ 判一下即可：

```

1 // h 只维护唯一记录会用到的两个端点，但同端点仍自然计算两次。
2 vector<int> h(n + 1, 0);
3 for (int d = 1; d <= k; d++) {
4     h[event[d]]++;
5     // 不变量：唯一记录的状态由当前 h[u]+h[v] 完全决定。
6     int answer = (h[u] + h[v] >= a[t] ? 1 : 0);
7     // 边界与答案：阈值在本次增加后达到时，当天就输出 1。
8     cout << answer << '\n';
9 }
```

复杂度 $O(K)$ 。这一档最直接地体现了「答案随事件推进单调不减」：一旦输出了 1，后面就全是 1。

为什么不能推广： Q 条记录时它要做 Q 遍，退化成 $O(KQ)$ 。

【子任务 3：所有记录都有 $i = j$ （15 分，测试点 06~08）】

特殊性质 B 保证每条记录都是 (i, i, k) ，条件变成 $2h_i \geq a_k$ ，也就是

$$h_i \geq \left\lceil \frac{a_k}{2} \right\rceil = \left\lfloor \frac{a_k + 1}{2} \right\rfloor.$$

记这个数为 $rank$ 。 h_i 每被增加一次就涨 1，所以它第一次达到 $rank$ 的那次事件，就是 $days[i]$ 里的第 $rank$ 个元素（下标从 1 数起）。如果 $days[i]$ 压根没有那么多个元素，这条记录一辈子都不会触发。

```

1 // born[d] 统计第 d 次事件后首次满足 2*h_i >= a_t 的记录。
2 // 关键转移：第 rank 次更新所在日期就是这条记录的首次触发日。
3 vector<int> born(k + 2, 0);
4 bool valid = true;
5 for (int z = 0; z < q; z++) {
6     int u, v, t;
```

```

7         cin >> u >> v >> t;
8         if (u != v) valid = false;
9         if (u == v) {
10             int rank = (a[t] + 1) / 2;
11             if ((int)days[u].size() >= rank) born[days[u][rank -
↪ 1]]++;
12         }
13     }

```

算出首次触发事件号之后，剩下的就是前缀和：

```

1     int answer = 0;
2     for (int d = 1; d <= k; d++) {
3         // 每次增加使  $2 * h_i$  增加 2，第  $rank$  次增加正好给出首次触发日。
4         answer += born[d];
5         // 边界与答案：不存在所需第  $rank$  次增加的记录永远不进入累计值。
6         cout << answer << '\n';
7     }

```

$(a[t] + 1) / 2$ 是整数上取整的标准写法，比 $\text{ceil}(a[t] / 2.0)$ 稳妥——后者要过一遍浮点，在大数上有精度隐患。

复杂度 $O(K + Q)$ 。满分做法要为每条记录多做一次二分，是 $O(K + Q \log K)$ ，比这一档多一个对数。这一档已经把「先算首次触发事件号、再前缀和」的框架搭好了，唯一没解决的是 $i \neq j$ 。

为什么不能推广： $i \neq j$ 时门槛压在两个计数器的和上，「第 $rank$ 次增加」没法在单个 $days$ 表里读出来。

【满分做法：合并两张表求第 $need$ 小（60 分，测试点 09~20）】

回到题意理解那张图里的那句话： $h_u + h_v$ 每次事件最多抬高一级，而且它抬级的那些事件，恰好就是 $days[u]$ 与 $days[v]$ 里的全部元素合起来（同一次事件不可能既属于 u 又属于 v ，因为一次事件只动一个计数器，所以两张表里的编号互不相同，合起来是一列各不相同的事件号）。

于是可以直接写出结论：把 $days[u]$ 和 $days[v]$ 归并成一个有序序列， $h_u + h_v$ 第 r 次抬级、也就是它第一次达到 r 的那次事件，就是这个序列里的第 r 小。记 $need = a_k$ ，则

首次触发事件号 = $(days[u] \cup days[v])$ 中第 $need$ 小的元素。

若两张表加起来都不足 $need$ 个元素，这条记录永不触发。

拿样例的记录 $(1, 2, 2)$ 验一遍: $days[1] = \{4\}$ 、 $days[2] = \{1, 2\}$, 合并后是 $1, 2, 4$, $need = a_2 = 3$, 第 3 小是 4 —— 与前面那张阶梯图读出来的完全一致。再看记录 $(3, 5, 3)$: $days[3] = \emptyset$ 、 $days[5] = \{3\}$, 合并后是 3 , $need = a_3 = 1$, 第 1 小是 3, 所以它在第 3 次事件之后首次触发。而记录 $(1, 2, 1)$ 的 $need = a_1 = 4$, 两表合起来只有 3 个元素, 永不触发; 记录 $(1, 5, 2)$ 的 $need = 3$, 两表合起来只有 $\{3, 4\}$ 两个元素, 也永不触发。四条记录合起来正好给出 $0, 0, 1, 2$ 。

剩下的问题就是: **怎么快速求两个有序表的第 $need$ 小**。真把它们归并出来是 $O(|days[u]| + |days[v]|)$ 的, 最坏情况下单条记录就要 $O(K)$, Q 条记录合计 $O(QK)$, 等于没优化。正确的做法是二分**切分点**: 设从 $days[u]$ 里取前 x 个、从 $days[v]$ 里取前 y 个, 令 $x + y = need$, 那么只要这两段的元素恰好就是全体中最小的 $need$ 个, 答案就是它们的最大值。

判断「恰好是最小的 $need$ 个」只需要看四个边界值: 左边取到的最后一个必须不大于右边没取的第一个, 两个方向都要成立。

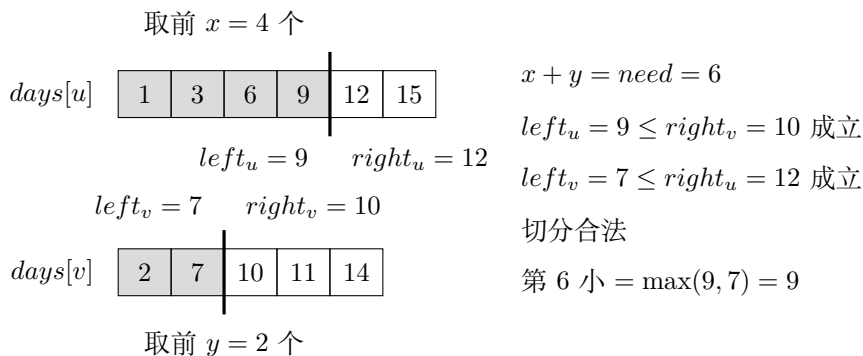


图 6: 在两个有序表里找第 $need = 6$ 小。灰底格子是被取走的前 $x + y = 6$ 个元素, 白底格子是剩下的; 粗竖线是切分位置, 紧挨着它标出的四个边界值交叉比较都成立, 说明取走的正是全体最小的 6 个, 第 6 小就是两侧最后一个里的较大者 9。

图里的切分是合法的, 可以自己数一下核对: 两表合并后的升序是 $1, 2, 3, 6, 7, 9, 10, 11, 12, 14, 15$, 前 6 个是 $1, 2, 3, 6, 7, 9$, 第 6 小确实是 9, 而灰底格子里装的正是 $\{1, 3, 6, 9\} \cup \{2, 7\}$, 一个不多一个不少。

如果某个方向不成立, 比如 $left_u > right_v$, 说明 $days[u]$ 这边取多了, 把 x 调小; 反之把 x 调大。这就是一个标准的二分, $O(\log K)$ 次判断即可收敛。代码里还先把 x 的搜索范围收紧到 $[\max(0, need - |days[v]|), \min(need, |days[u]|)]$, 保证 $y = need - x$ 始终落在合法区间内。

还剩一支要单独处理: $u = v$ 时两张表其实是同一张, 合并会把每个事件数两遍, 上面的结论套不上。这类记录沿用子任务 3 的做法——首次触发事件号就是 $days[u]$ 里的第 $\lceil need/2 \rceil$ 个元素。下面这段代码把这一支放在最前面, $u \neq v$ 时才进入二分。

```

1  if (u == v) {
2      int rank = (need + 1) / 2;
3      if ((int)days[u].size() >= rank) day = days[u][rank - 1];

```

```

4         } else if ((int)days[u].size() + (int)days[v].size() >= need)
    ↪ {
5             // 不变量: 切分左右两表的前 need 个日期, 左侧值都不大于右侧值。
6             int lo = max(0, need - (int)days[v].size());
7             int hi = min(need, (int)days[u].size());
8             while (lo <= hi) {
9                 int x = (lo + hi) / 2;
10                int y = need - x;
11                int left_u = (x == 0 ? 0 : days[u][x - 1]);
12                int right_u = (x == (int)days[u].size() ? k + 1 : days
    ↪ [u][x]);
13                int left_v = (y == 0 ? 0 : days[v][y - 1]);
14                int right_v = (y == (int)days[v].size() ? k + 1 : days
    ↪ [v][y]);
15                if (left_u <= right_v && left_v <= right_u) {
16                    day = max(left_u, left_v);
17                    break;
18                }
19                if (left_u > right_v) hi = x - 1;
20                else lo = x + 1;
21            }
22        }

```

$x = 0$ 或 $x = |days[u]|$ 这两个端点要特别处理: 前者表示 $days[u]$ 一个都不取, 此时「左边最后一个」不存在, 用 0 代替 (比任何真实事件号都小); 后者表示全取, 此时「右边第一个」不存在, 用 $k + 1$ 代替 (比任何真实事件号都大)。这两个哨兵让边界情形和一般情形共用同一段判断, 不必写额外分支。

首次触发事件号全部算完之后, 累计输出:

```

1     // 边界与答案: 先加入当天新触发记录, 再输出当前累计触发总数。
2     int answer = 0;
3     for (int d = 1; d <= k; d++) {
4         answer += born[d];
5         cout << answer << '\n';
6     }

```

复杂度: 读入 $O(K + Q)$, 每条记录一次二分 $O(\log K)$, 前缀和 $O(K)$, 合计 $O(K + Q \log K)$ 。 $K = Q = 2 \times 10^5$ 时约 $2 \times 10^5 \times 18 \approx 3.6 \times 10^6$ 次判断, 两秒之内非常宽裕。空间是 $days$ 的 $O(K)$ 加 $born$ 的 $O(K)$ 。

顺带一提，因为答案的输出是 K 行、 Q 与 K 都到 2×10^5 ，读写量不小，代码开头的 `ios::sync_with_stdio(false)` 与 `cin.tie(0)` 不能省，去掉它们在最大数据上会明显变慢。

【易错点】

- $i = j$ 要算两次。条件是 $2h_i \geq a_k$ 而不是 $h_i \geq a_k$ ，对应的名次是 $\lceil a_k/2 \rceil$ 。写漏这个分支，性质 B 那一档会直接全错。
- 相同的记录行不能去重。题面明说每行独立计数，把三元组塞进集合去重会少算。
- 永不触发的记录要挡在外面。两表元素总数不足 $need$ 时不能往 `born` 里写，代码用 $day = k + 1$ 加上 `if (day <= k)` 来拦，漏了会数组越界或多算。
- 上取整别用浮点。 $\lceil a/2 \rceil$ 写成 $(a+1)/2$ ；`ceil(a/2.0)` 在大数上有精度风险。
- 二分的哨兵不能省。 $x = 0$ 与 $x = |days[u]|$ 时四个边界值里有两个不存在，必须用 0 和 $k + 1$ 顶上，否则会访问到数组外。
- 先加后问。每次事件是「先把 h_{x_d} 加一，再统计」，顺序写反会让每一行输出都慢一拍。

数据抢救 题解

【题意理解】

n 个数据卷，第 i 卷有一个索引块和 n_i 个数据块，每个块都有自己的读取时间和信息价值。规则只有一条：**要读这一卷的任何数据块，必须先读这一卷的索引块**；索引块本身没有前提，也可以只读索引块不读数据块。在总读取时间不超过 T 的前提下，求能恢复的信息价值总和的最大值。

先和最基础的 01 背包对照一下。如果没有那条依赖，这就是一道标准的 01 背包： $\sum n_i + n$ 个物品，容量 T ，每个物品选或不选。依赖的存在让「选」这个动作不再自由——数据块能不能选，取决于索引块选没选。

但依赖的结构很简单：每一卷就是一棵深度只有 2 的小树，根是索引块、叶子是数据块，卷与卷之间没有任何关系。由此得到处理办法：**按卷来做决策**。对第 i 卷，整体只有两种可能——

1. 整卷跳过（索引块和数据块都不读）；
2. 读索引块，然后在这一卷的数据块里自由地做一次 01 背包。

只要能把第 2 种情况打包成「一件可以放进背包的复合物品」，外层就退化成对 n 个卷做一次普通的「选或不选」。

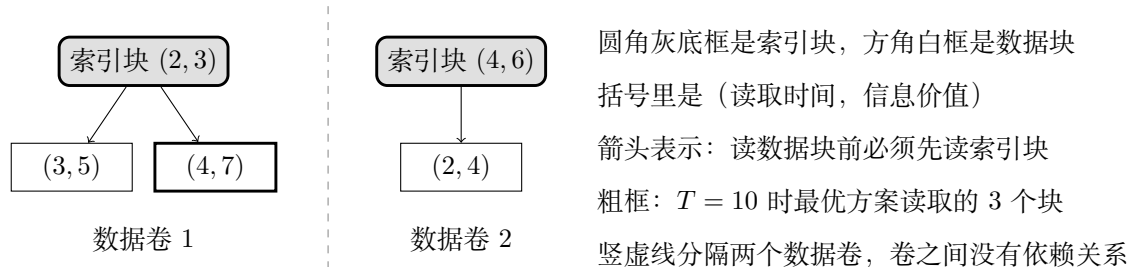


图 7：依赖结构：每一卷是一棵深度 2 的小树，根是索引块。全文用这个 $T = 10$ 的例子做演示，它的最优方案是「卷 1 索引块 + 卷 1 的 (4, 7) + 卷 2 索引块」，用时 $2 + 4 + 4 = 10$ ，价值 $3 + 7 + 6 = 16$ 。

样例 1 本身是个更简单的情形：三个数据卷都满足 $n_i = 0$ ，索引块分别是 (2, 3), (3, 6), (5, 8)，全部读取正好用满 $T = 10$ ，价值 $3 + 6 + 8 = 17$ 。

四个子任务的区别，就在于用什么办法处理「整卷跳过 / 读索引块再挑数据块」这个二选一。

【子任务 1：枚举所有块的子集（15 分，测试点 01~03）】

这一档 $n \leq 5$ 、 $n_i \leq 3$ ，块的总数最多 $5 \times (1 + 3) = 20$ 个，可以把每个块选或不选的所有 2^{20} 种组合全枚举一遍。

实现上把所有块摊平进一个数组，同时用 `need[x]` 记下第 x 个块依赖哪个索引块（索引块自己记 -1 ）。对每个 `mask`，一边累加时间与价值，一边检查每个被选中的块的前置索引块是否也在 `mask` 里：

```

1  // 状态含义：每个 mask 就是一整套选择方案，need[x] 记录它依赖的索引块。
2  long long ans = 0;
3  unsigned long long total = 1ULL << cost.size();
4  for (unsigned long long mask = 0; mask < total; ++mask) {
5      int used = 0;
6      long long sum = 0;
7      bool ok = true;
8      for (int j = 0; j < (int)cost.size(); ++j) {
9          if ((mask >> j) & 1ULL) {
10             used += cost[j];
11             sum += value[j];
12             if (need[j] >= 0 && ((mask >> need[j]) & 1ULL) == 0)
13                 ok = false;
14             }
15         }
16         // 转移不变量：只有容量够、且前置索引块全都选了，才允许更新。
17         if (ok && used <= T) ans = max(ans, sum);
18     }
19     // 边界与答案：允许空集，ans 保存最终最大值。
20     cout << ans << '\n';

```

复杂度 $O(2^B \cdot B)$ ，其中 B 是块总数，这里 $2^{20} \times 20 \approx 2 \times 10^7$ ，一秒内能过。 B 再大一点就彻底不行了——完整范围下 B 可达 $100 \times 501 \approx 5 \times 10^4$ 。

为什么不能推广：它把「哪些块被选」这个指数级的信息完整地枚举了一遍，而背包问题真正需要的只有「用了多少时间、拿了多少价值」这两个数。

【子任务 2：只有一个数据卷（15 分，测试点 04~06）】

特殊性质 A 保证 $n = 1$ 。这时候外层的「选哪些卷」消失了，只剩两种方案：一个块都不读（价值 0），或者读下这唯一的索引块、再在剩下的 $T - t_{10}$ 时间里对数据块做 01 背包。

```

1 // 状态含义：dp[w] 是付掉索引块之后，容量 w 以内数据块的最大价值。
2 if (base_time > T) {
3     cout << 0 << '\n';
4     return 0;
5 }
6 vector<long long> dp(T - base_time + 1, 0);
7 // 转移不变量：倒序更新保证每个数据块最多用一次。
8 for (int j = 0; j < cnt; ++j) {
9     int t = items[j].first;
10    int c = items[j].second;
11    for (int w = T - base_time; w >= t; --w) {
12        dp[w] = max(dp[w], dp[w - t] + c);
13    }
14 }
15 // 边界与答案：什么都不选得 0；否则要把索引块的价值补上一次。
16 cout << max(0LL, dp[T - base_time] + base_value) << '\n';

```

有三处细节值得看。其一，背包的容量是 $T - t_{10}$ 而不是 T ——索引块的时间已经先付掉了。其二，如果 $t_{10} > T$ ，索引块本身就装不下，直接输出 0。其三，最后那句 $\max(0LL, \dots)$ 保证「什么都不读」这个合法方案不会被漏掉。

内层的 01 背包写法是标准的：容量倒序枚举，保证每个数据块最多被用一次。如果写成正序，同一个数据块会在一轮里被重复放入，变成完全背包，答案偏大。

复杂度 $O(n_1 T)$ ，最大 $500 \times 500 = 2.5 \times 10^5$ 。

为什么不能推广： $n > 1$ 时，各卷要在同一个总读取时间上限 T 里互相争抢，不能各算各的。

【子任务 3：所有卷都没有数据块（15 分，测试点 07~09）】

特殊性质 B 保证 $n_i = 0$ ，依赖关系全部消失，题目退化成最朴素的 01 背包： n 个索引块，选或不选。

```

1 // 状态含义：dp[w] 是容量 w 以内，只取索引块能得到的最大价值。
2 vector<long long> dp(T + 1, 0);
3 // 转移不变量：倒序更新保证每个索引块只被取一次（01 背包）。
4 for (int i = 0; i < n; ++i) {

```

```

5     int t = bases[i].first;
6     int c = bases[i].second;
7     for (int w = T; w >= t; --w) {
8         dp[w] = max(dp[w], dp[w - t] + c);
9     }
10 }
11 // 边界与答案：允许容量用不满，所以 dp[T] 就是最终最大值。
12 cout << dp[T] << '\n';

```

$dp[w]$ 的含义是「总读取时间不超过 w 时的最大价值」。因为允许时间用不满， dp 数组天然是不减的，所以最后直接读 $dp[T]$ 即可，不需要在所有 w 上取最大值。样例 1 正是这一档：三个索引块 (2,3), (3,6), (5,8) 全读恰好用满 10，答案 17。

复杂度 $O(nT) = 100 \times 500 = 5 \times 10^4$ 。

为什么不能推广：一旦某卷有数据块，「选了索引块之后还能继续往里塞东西」这件事就没法用单个 01 物品表示了。

【满分做法：按卷分层的依赖背包（55 分，测试点 10~20）】

把前三档的想法拼起来：外层像子任务 3 那样对 n 个卷做「选或不选」，内层像子任务 2 那样对本卷的数据块做 01 背包。关键是把这两层正确地衔接。

设 $dp[w]$ 表示前面若干卷处理完之后，总读取时间不超过 w 时的最大价值。处理第 i 卷时开一个临时数组 $take[w]$ ，它的含义是：

| $take[w]$ = 总读取时间不超过 w ，并且已经确定读了第 i 卷的索引块时的最大价值。

初始化就是把索引块的代价先付掉——从 dp 里平移过来：

```

1     // 状态含义：take[w] 表示已经选了本卷索引块之后，容量 w 对应的最大
    ↪ 价值。
2     vector<long long> take(T + 1, bad);
3     for (int w = base_time; w <= T; ++w) {
4         take[w] = dp[w - base_time] + base_value;
5     }

```

$w < t_{i0}$ 的那些格子装不下索引块，被留作 bad （一个极小的负数）。用 bad 而不是 0 是必须的：如果这些格子写 0，就等于承认「没读索引块也能进入 $take$ 层」，后面的数据块会被非法地放进去。

付过索引块之后，本卷的数据块就自由了，对 $take$ 做一次标准的 01 背包：

```

1     // 转移不变量：倒序更新保证每个数据块只是一次 01 选择。
2     for (int j = 0; j < cnt; ++j) {

```

```

3         int cost = items[j].first;
4         int value = items[j].second;
5         for (int w = T; w >= cost; --w) {
6             if (take[w - cost] != bad) {
7                 take[w] = max(take[w], take[w - cost] + value);
8             }
9         }
10    }

```

`if (take[w - cost] != bad)` 这个判断守住了依赖：只有那些确实已经付过索引块的状态才允许继续扩展。

最后把两种决策合并——`dp[w]` 保持原样表示「整卷跳过」，`take[w]` 表示「读了这一卷」，取较大者：

```

1         // 边界与答案：保留 dp 表示跳过本卷，take 则已经付过本卷索引块的代
    ↪ 价。
2         for (int w = 0; w <= T; ++w) {
3             dp[w] = max(dp[w], take[w]);
4         }

```

注意 `take` 是每卷新建的，不能在卷之间复用，否则上一卷的索引块会被误认为已经付过。

下面把插图里那个例子的第一卷完整走一遍。 $T = 10$ ，卷 1 是索引块 (2,3) 加数据块 (3,5)、(4,7)。

w	0	1	2	3	4	5	6	7	8	9	10	
旧 dp	0	0	0	0	0	0	0	0	0	0	0	处理本卷之前的结果
$take$ 初值	$-\infty$	$-\infty$	3	3	3	3	3	3	3	3	3	旧 dp 右移 2 格再加 3
放入 (3,5)	$-\infty$	$-\infty$	3	3	3	8	8	8	8	8	8	$w \geq 5$ 的格子被顶高
放入 (4,7)	$-\infty$	$-\infty$	3	3	3	8	10	10	10	15	15	$w \geq 6$ 的格子被顶高
新 dp	0	0	3	3	3	8	10	10	10	15	15	与旧 dp 逐格取较大者

灰底的 $-\infty$ 就是代码里的 `bad`：这两格连索引块都装不下；粗框圈出放入该数据块后被抬高的格子。

图 8：处理第一卷时 `dp` 与 `take` 的五个快照 ($T = 10$ ，卷 1 的索引块是 (2,3)，两个数据块是 (3,5) 与 (4,7))。 `take` 初值由旧 `dp` 整体右移 2 格再加 3 得到；两个数据块依次做倒序 01 背包；最后 `take` 与旧 `dp` 逐格取较大者写回 `dp`。

图里几个数可以自己核一核：`take[9]` 最终是 15，对应「索引块 + 两个数据块」，

用时 $2 + 3 + 4 = 9$ 、价值 $3 + 5 + 7 = 15$ ；`take[6]` 是 10，对应「索引块 $+(4, 7)$ 」，用时 6、价值 10；而 `take[5]` 只有 8，因为 $(4, 7)$ 塞不进剩下的 3 单位时间，只能用 $(3, 5)$ 。合并到 `dp` 时前两格恢复成 0（整卷跳过），其余格子都被 `take` 顶上去了。

接着处理第二卷（索引块 $(4, 6)$ 、数据块 $(2, 4)$ ），`take[10]` 的初值是 `dp[6] + 6 = 10 + 6 = 16`；再把数据块 $(2, 4)$ 放进去只能得到 `take[8] + 4 = 9 + 4 = 13`，反而更差，所以 `take[10]` 仍是 16。最终 `dp[10] = max(15, 16) = 16`，正是插图里说的那个最优方案。

复杂度：外层每卷的初始化与合并都是 $O(T)$ ，内层数据块的 01 背包是 $O(n_i T)$ ，合计

$$O\left(nT + T \sum_i n_i\right).$$

最大规模下 $\sum n_i \leq 100 \times 500 = 5 \times 10^4$ ，乘上 $T = 500$ 是 2.5×10^7 次基本运算，一秒之内很宽裕。空间只有两个长度 $T + 1$ 的数组， $O(T)$ 。

价值用 `long long`：单块价值上限 10^5 ，块数上限约 5×10^4 ，理论总和可达 5×10^9 ，已经超过 `int`。虽然受 $T \leq 500$ 限制实际选不了那么多块，但把类型放宽不会增加任何开销。`bad` 取 -2^{60} 也是同一个考虑——它要足够小，小到「加上若干个价值之后仍然是负数」，这样它永远不会在 `max` 里胜出。

【易错点】

- **take** 的无效格必须是极小值，不能是 0。写 0 等于允许「没读索引块就读数据块」，依赖关系直接失效。
- **take** 每卷都要重建。跨卷复用会让前一卷的索引块被当成已付。
- 内层 01 背包要倒序。正序会把一个数据块重复选入，变成完全背包。
- 索引块可以单独读。`take` 的初值本身就是「只读索引块」的方案，不要漏掉它，也不要要求至少读一个数据块。
- 别忘了整卷跳过。合并时是 `dp[w] = max(dp[w], take[w])`，直接赋值成 `take[w]` 会强迫每一卷都被读。
- $n_i = 0$ 的卷不要特判出错。它没有数据块行，内层循环跑零次，逻辑照常成立；若在读入时假设至少有一行，会读串整个输入。
- 价值用 `long long`。块数与单价的乘积会超出 `int`，`bad` 也要取得足够小。